CS 436 Front End Design Lab: Sessions 1, 2, 3, 5, 6, 7

All JavaScript goes inside `<script> ... </script>` in an HTML page. `prompt()` takes input from the user. Output is shown as it appears in the console.

Session 1: Introduction to JS

1.1 Write basic JavaScript programs to explore programming constructs, functions and closures.

```
        // function
        function add(a, b) {
          return a + b;
        }

        // loop + condition
        for (let i = 1; i <= 5; i++) {
          if (i % 2 == 0) console.log(i + " is even");
        }

        // closure: inner function remembers count
        function counter() {
          let count = 0;
          return function () {
            count++;
            return count;
          };
        }
        let c = counter();

        console.log("Sum =", add(4, 6));
        console.log(c(), c(), c());
```

Output:

```
2 is even
4 is even
Sum = 10
1 2 3
```

1.2 Display a welcome message using alert() in HTML.

```
        <html>
        <body>
          <h2>My Page</h2>
          <script>
            alert("Welcome to Front End Design Lab!");
          </script>
        </body>
        </html>
```

Output: a popup box saying `Welcome to Front End Design Lab!`

1.3 Use literals, identifiers and variable declarations.

```
        // identifiers: name, age, pi, isStudent, marks, student
        var name = "Amit";              // string literal
        let age = 21;                   // number literal
        const pi = 3.14;                // constant
        let isStudent = true;           // boolean literal
        let marks = [80, 90, 75];       // array literal
        let student = { roll: 101 };    // object literal

        console.log(name, age, pi, isStudent);
        console.log(marks, student);
```

Output:

```
Amit 21 3.14 true
[ 80, 90, 75 ] { roll: 101 }
```

1.4 Declare and print all primitive data types.

```
        let num = 42;
        let str = "Hello";
        let bool = true;
        let undef;
        let empty = null;
        let big = 12345678901234567890n;
        let sym = Symbol("id");

        console.log(num, typeof num);
        console.log(str, typeof str);
        console.log(bool, typeof bool);
        console.log(undef, typeof undef);
        console.log(empty, typeof empty);
        console.log(big, typeof big);
        console.log(sym.toString(), typeof sym);
```

Output:

```
42 number
Hello string
true boolean
undefined undefined
null object
12345678901234567890n bigint
Symbol(id) symbol
```

Session 2: Variables, Operators, Control Flow

2.1 Accept a number and insert dashes (-) between each pair of even digits. Input 025468 gives 0-254-6-8.

```
        let num = prompt("Enter a number");   // input: 025468
        let result = num[0];

        for (let i = 1; i < num.length; i++) {
          if (num[i] % 2 == 0 && num[i - 1] % 2 == 0) {
            result += "-";
          }
          result += num[i];
        }
        console.log(result);
```

Output:

```
0-254-6-8
```

2.2 Demonstrate var, let and const scopes.

```
        function scopeTest() {
          if (true) {
            var a = 10;      // function scoped
            let b = 20;      // block scoped
            const c = 30;    // block scoped
            console.log("Inside block:", a, b, c);
          }
          console.log("Outside block, var a =", a);
          try {
            console.log(b);
          } catch (e) {
            console.log("Outside block, b is not accessible");
          }
        }
        scopeTest();
```

Output:

```
Inside block: 10 20 30
Outside block, var a = 10
Outside block, b is not accessible
```

2.3 Create a calculator for +, -, *, /, %, **.

```
        let a = Number(prompt("Enter first number"));    // input: 6
        let b = Number(prompt("Enter second number"));   // input: 3
        let op = prompt("Enter operator");               // input: **
        let res;

        switch (op) {
          case "+":  res = a + b;  break;
          case "-":  res = a - b;  break;
          case "*":  res = a * b;  break;
          case "/":  res = a / b;  break;
          case "%":  res = a % b;  break;
          case "**": res = a ** b; break;
          default:   res = "Invalid operator";
        }
        console.log(a + " " + op + " " + b + " = " + res);
```

Output:

```
6 ** 3 = 216
```

2.4 Use typeof to check data types.

```
        console.log(typeof 25);
        console.log(typeof "JS");
        console.log(typeof false);
        console.log(typeof undefined);
        console.log(typeof null);
        console.log(typeof [1, 2]);
        console.log(typeof { a: 1 });
        console.log(typeof function () {});
```

Output:

```
number
string
boolean
undefined
object
object
object
function
```

2.5 Check if a number is even or odd using if-else.

```
        let n = Number(prompt("Enter a number"));   // input: 7

        if (n % 2 == 0) {
          console.log(n + " is Even");
        } else {
          console.log(n + " is Odd");
        }
```

Output:

```
7 is Odd
```

2.6 Use switch-case for a grading system.

```
        let marks = Number(prompt("Enter marks"));   // input: 82
        let grade;

        switch (true) {
          case marks >= 90: grade = "A"; break;
          case marks >= 75: grade = "B"; break;
          case marks >= 60: grade = "C"; break;
          case marks >= 40: grade = "D"; break;
          default:          grade = "F";
        }
        console.log("Marks: " + marks + ", Grade: " + grade);
```

Output:

```
Marks: 82, Grade: B
```

2.7 Print primes from 1 to 100 using a for loop.

```
        let primes = "";

        for (let n = 2; n <= 100; n++) {
          let isPrime = true;
          for (let i = 2; i * i <= n; i++) {
            if (n % i == 0) {
              isPrime = false;
              break;
            }
          }
          if (isPrime) primes += n + " ";
        }
        console.log(primes);
```

Output:

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

Session 3: Objects and Operations

3.1 Create and access object properties.

```
        let student = {
          name: "Riya",
          roll: 23,
          branch: "CSE"
        };

        console.log(student.name);        // dot notation
        console.log(student["branch"]);   // bracket notation
        student.cgpa = 8.7;               // add a property
        console.log(student);
```

Output:

```
Riya
CSE
{ name: 'Riya', roll: 23, branch: 'CSE', cgpa: 8.7 }
```

3.2 Use accessors and mutators.

```
        let person = {
          firstName: "Amit",
          lastName: "Kumar",

          get fullName() {                 // accessor (getter)
            return this.firstName + " " + this.lastName;
          },
          set fullName(value) {            // mutator (setter)
            let parts = value.split(" ");
            this.firstName = parts[0];
            this.lastName = parts[1];
          }
        };

        console.log(person.fullName);
        person.fullName = "Rahul Sharma";
        console.log(person.firstName, person.lastName);
```

Output:

```
Amit Kumar
Rahul Sharma
```

3.3 Iterate object properties using for-in.

```
        let car = { brand: "Honda", model: "City", year: 2022 };

        for (let key in car) {
          console.log(key + " : " + car[key]);
        }
```

Output:

```
brand : Honda
model : City
year : 2022
```

3.4 Use static methods and nullify an object.

```
class MathUtil {
  static square(x) {
    return x * x;
  }
}
console.log("Square of 5 =", MathUtil.square(5));   // called on class

let obj = { id: 1, name: "Test" };
console.log("Before:", obj);
obj = null;                      // object is now eligible for garbage collection
console.log("After:", obj);
```

Output:

```
Square of 5 = 25
Before: { id: 1, name: 'Test' }
After: null
```

Session 5: Functions and Arrays

5.1 Function to get the last element of an array. Passing n returns the last n elements.

```
function last(arr, n) {
  if (n == undefined) {
    return arr[arr.length - 1];
  }
  return arr.slice(-n);
}

console.log(last([7, 9, 0, -2]));
console.log(last([7, 9, 0, -2], 3));
```

Output:

```
-2
[ 9, 0, -2 ]
```

5.2 Write a recursive factorial function.

```
function factorial(n) {
  if (n <= 1) return 1;          // base case
  return n * factorial(n - 1);   // recursive call
}
console.log("5! =", factorial(5));
```

Output:

```
5! = 120
```

5.3 Sum elements of an array using a loop.

```
let arr = [10, 20, 30, 40];
let sum = 0;

for (let i = 0; i < arr.length; i++) {
  sum += arr[i];
}
console.log("Sum =", sum);
```

Output:

```
Sum = 100
```

5.4 Declare an array using a literal and a constructor.

```
let fruits = ["Apple", "Mango", "Banana"];        // literal
let nums = new Array(1, 2, 3, 4);                 // constructor

console.log(fruits);
console.log(nums);
```

Output:

```
[ 'Apple', 'Mango', 'Banana' ]
[ 1, 2, 3, 4 ]
```

5.5 Return an updated array with r right rotations on an array of integers a.

```
function rotateRight(a, r) {
  r = r % a.length;
  return a.slice(-r).concat(a.slice(0, a.length - r));
}

console.log(rotateRight([1, 2, 3, 4, 5], 2));
```

Output:

```
[ 4, 5, 1, 2, 3 ]
```

Session 6: Array Methods, Sorting, 2D Arrays

6.1 Find the most frequent item in an array.

```
let arr = [3, 'a', 'a', 'a', 2, 3, 'a', 3, 'a', 2, 4, 9, 3];
let count = {};
let maxItem = arr[0], maxCount = 0;

for (let item of arr) {
  count[item] = (count[item] || 0) + 1;
  if (count[item] > maxCount) {
    maxCount = count[item];
    maxItem = item;
  }
}
console.log(maxItem + " ( " + maxCount + " times )");
```

Output:

```
a ( 5 times )
```

6.2 Add items to a blank array and display them.

```
let list = [];

list.push("HTML");
list.push("CSS");
list.push("JavaScript");

for (let i = 0; i < list.length; i++) {
  console.log((i + 1) + ". " + list[i]);
}
```

Output:

```
1. HTML
2. CSS
3. JavaScript
```

6.3 Remove duplicate items from an array (ignore case sensitivity).

```
let arr = ["Apple", "mango", "APPLE", "Banana", "MANGO"];
let seen = [];
let result = [];

for (let item of arr) {
  let lower = item.toLowerCase();
  if (!seen.includes(lower)) {
    seen.push(lower);
    result.push(item);
  }
}
console.log(result);
```

Output:

```
[ 'Apple', 'mango', 'Banana' ]
```

6.4 Use map(), filter(), reduce().

```
let nums = [1, 2, 3, 4, 5, 6];

let doubled = nums.map(x => x * 2);
let evens   = nums.filter(x => x % 2 == 0);
let total   = nums.reduce((sum, x) => sum + x, 0);

console.log("map    :", doubled);
console.log("filter :", evens);
console.log("reduce :", total);
```

Output:

```
map    : [ 2, 4, 6, 8, 10, 12 ]
filter : [ 2, 4, 6 ]
reduce : 21
```

6.5 Sort student marks in descending order.

```
let students = [
  { name: "Amit",  marks: 78 },
  { name: "Riya",  marks: 92 },
  { name: "Kiran", marks: 85 }
];

students.sort((a, b) => b.marks - a.marks);

for (let s of students) {
  console.log(s.name + " - " + s.marks);
}
```

Output:

```
Riya - 92
Kiran - 85
Amit - 78
```

6.6 Create and access a 2D array (matrix).

```
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

console.log("Element at [1][2] =", matrix[1][2]);

for (let i = 0; i < matrix.length; i++) {
  let row = "";
  for (let j = 0; j < matrix[i].length; j++) {
    row += matrix[i][j] + " ";
  }
  console.log(row);
}
```

Output:

```
Element at [1][2] = 6
1 2 3
4 5 6
7 8 9
```

Session 7: BOM (Navigator, Location, History, Screen)

7.1 Explore the object-oriented properties of JavaScript.

```
        class Animal {                          // class + encapsulation
          constructor(name) {
            this.name = name;
          }
          speak() {
            return this.name + " makes a sound";
          }
        }

        class Dog extends Animal {              // inheritance
          speak() {                             // polymorphism (overriding)
            return this.name + " barks";
          }
        }

        let a = new Animal("Cat");
        let d = new Dog("Tommy");
        console.log(a.speak());
        console.log(d.speak());
        console.log(d instanceof Animal);
```

Output:

```
Cat makes a sound
Tommy barks
true
```

7.2 Display browser info using navigator.

```
        <script>
          document.write("Browser: " + navigator.userAgent + "<br>");
          document.write("Language: " + navigator.language + "<br>");
          document.write("Platform: " + navigator.platform + "<br>");
          document.write("Online: " + navigator.onLine);
        </script>
```

Output (depends on your browser):

```
Browser: Mozilla/5.0 (Windows NT 10.0; Win64; x64) ... Chrome/129.0 ...
Language: en-US
Platform: Win32
Online: true
```

7.3 Change page using location.href and history.back().

```
        <button onclick="goPage()">Go to Google</button>
        <button onclick="goBack()">Go Back</button>

        <script>
          function goPage() {
            location.href = "https://www.google.com";
          }
          function goBack() {
            history.back();
          }
        </script>
```

Output: “Go to Google” opens google.com in the same tab. “Go Back” returns to the previous page in history.

7.4 Display screen resolution using screen.

```
        <script>
          document.write("Screen Width: " + screen.width + "<br>");
          document.write("Screen Height: " + screen.height + "<br>");
          document.write("Resolution: " + screen.width + " x " + screen.height);
        </script>
```

Output (depends on your screen):

```
Screen Width: 1920
Screen Height: 1080
Resolution: 1920 x 1080
```